

AI-Powered B-roll Search & Retrieval System

PS 3 | 9-hour work division | Krups: ML • Kavya: Backend • Siya: Frontend

Agree in hour 1: the API contract

Kavya's API returns one result shape so Siya can build the UI against mock data before the backend exists. If Streamlit can call the Python functions directly, skip FastAPI to save integration time.

```
{ "clip_id": "", "start": 12.0, "end": 18.0, "percent": 81,
  "caption": "", "thumbnail_url": "", "video_url": "", "quality_flag": null }
```

Script mode returns a list of {scene_index, sentence, results: [...]}.

Hour-by-hour plan

Hour	Krups (ML)	Kavya (Backend)	Siya (Frontend)
H1 Setup	Load CLIP ViT-B/32 and embed one sample video. Time CPU vs GPU . Embed test queries and calibrate the min/max for percent match.	Repo and folder structure, SQLite schema, API contract, zip validation (under 500 MB, valid zip).	Streamlit skeleton with Upload, Single Search and Script Mode tabs. Get Pexels API key and start the download script.
H2 Pipeline pieces	Frame extraction (1 per 2s under 30s clips, 1 per 3-4s above, max 15). Group 3-4 frames into averaged segment embeddings.	Upload handler: unzip to /data/videos/{job_id}/, loop over videos with try/except so corrupt files are skipped, write clip metadata to SQLite.	Upload page with progress bar (mock data). Result card: thumbnail, % match, timestamp, "Play from here" via HTML5 currentTime.
H3 Pipeline pieces	FAISS IndexFlatIP: build, save, load, with index position to segment_id mapping. Command-line search(query, k) returns raw scores.	Pure functions with tests on fake data: sentence splitter (skip under 3 words, cap 20), percent conversion, threshold filter (35%), dedup (max 2 segments per clip).	Layouts for both tabs, "No strong match found" state, truncation notice, scene-by-scene view. Keep curating the clip library.
H4 Merge	Index the real demo library, check speed, cut frames to 8-10 per clip if CPU is slow. Recalibrate min/max on real scores.	Wire the search pipeline: raw search, then percent, threshold, dedup. Script mode = that function in a loop.	Finish the library (50-80 clips, including ones for planned wow queries). Test a real zip upload.
H5 Checkpoint	Timestamp localization and thumbnail extraction at the matched timestamp (OpenCV).	Expose /upload, /search, /script endpoints. Serve thumbnails and videos.	Connect the UI to the real endpoints, replacing the mocks.
H6 Integration	BLIP captions for the top 5 results only. Then the quality flag (blur and darkness check) if time allows.	Fix integration bugs, handle edge cases (empty query, corrupt file, long script), keep API errors readable in the UI.	Show captions and quality flags on cards. Polish layout and loading states.
H7 Test	Run the 8 wow queries, tune thresholds and percent range, swap weak library clips.	Test script mode with 2-3 prepared scripts and fix splitter oddities.	Make the demo flow smooth. Check the wow queries look good on screen.
H8 Wrap-up	Write the "how it works" slide and honest-limitations notes.	Write the README (handled vs deferred lists).	Record a backup demo video in case the live demo fails.
H9 Buffer	Rehearse the demo. Fix only small bugs.	Final submission checks: repo clean, run instructions work from scratch.	Final rehearsal. Prepare answers to likely judge questions.

Checkpoints

- **End of H1:** CLIP runs, CPU speed is known, API contract is agreed. If CPU is too slow, cut frames per clip right then.
- **End of H4, at the latest H5:** one working end-to-end search function, even if command-line only. Hard rule: if it is not done, **cut script mode and ship single-query search only**.
- **End of H6:** the full flow works in the UI on the real library. After this, only bug fixes and polish.

Notes

- **Krups is on the critical path** through H5. Kavya and Siya work against mock data and pure functions so they never wait on him.
- Only start Should-Haves (captions, quality flag) once the Must-Haves work end to end.
- **Siya starts the Pexels download in H1.** Curating clips that match the wow queries takes longer than it seems.
- Kavya and Krups pair for 30 minutes at the start of H4, where the two halves of the search function meet.